Event Management System Database Project

GitHub Repository: https://github.com/ksenia180507/Event-Management-System-Database

Video link: Loom (https://www.loom.com/share/e0025d46cb0345388023d3a0a8be4766)

(Event_Management_Database_Overview_and_Queries.mp4 – link to OneDrive in case Loom is not working)

Student name: Keseniia Goncharova

Student ID: GH1050782

Module: B103 Database & Big Data

Table of contents

1.0 Introduction

The purpose of this project is to design and implement a database for an Event Management System that simplifies and supports the planning, organization, and execution of events. The main objectives of the project are to create a relational database normalized to Third Normal Form (3NF), implement tables with primary and foreign key constraints, store and manage event-related data, and at the end demonstrate SQL queries (e.g CRUD operations) for effective data management and retrieval.

This system allows it to handle large amounts of information, to provide different types of users certain access. For example, organizers can create and manage events, set up venues, and schedule sessions. Participants can register for events and purchase tickets. Speakers can manage their sessions.

2.0 System design

The database includes multiple entities such as: Roles, Users, Organizers, Participants, Speakers, Venues, Events, Categories, Sessions, Tickets, Registrations, Event_category and Session_speaker. The system has three types of relationships: one-to-one, one-to-many, many-to-many.

One-to-one includes relationships between users and participants, organizers, speakers and organizers (e.g one User → one Organizer, one User → one Speaker etc.)

One-to-many are:

- One role → many Users
- One Venue → many Events
- One Organizer → many Events
- One Event → many Sessions
- One Event → many Tickets
- One Participant → many Registrations

Many-to-many:

- Events → Categories (using Event_category)
- Sessions → Speakers (using Session_speaker)

Table name	Primary key	Attributes	Foreign key(s)
Users	user_id	first_name, last_name, email, pnone_number	role_id
Roles	role_id	role_name	NULL
Organizers	organizer_id	NULL	user_id
Participants	participant_id	date_of_birth	user_id
Speakers	speaker_id	field_expertise	user_id
Events	event_id	event_name, start_date, end_date,	venue_id, organizer_id
Categories	category_id	category_name	NULL
Venues	venue_id	venue_name, country, city, street_address, max_capacity	NULL
Sessions	session_id	session_title, start_time, end_time	event_id
Tickets	ticket_id	ticket_type, price, amount	event_id
Registrations	registration_id	purchase_date	participant_id, event_id, ticket_id
Event_category	event_category_id	NULL	event_id, category_id

Sessions_speakers	sess_speak_id	NULL	session_id, speaker_id

Table 1: Attributes of the database

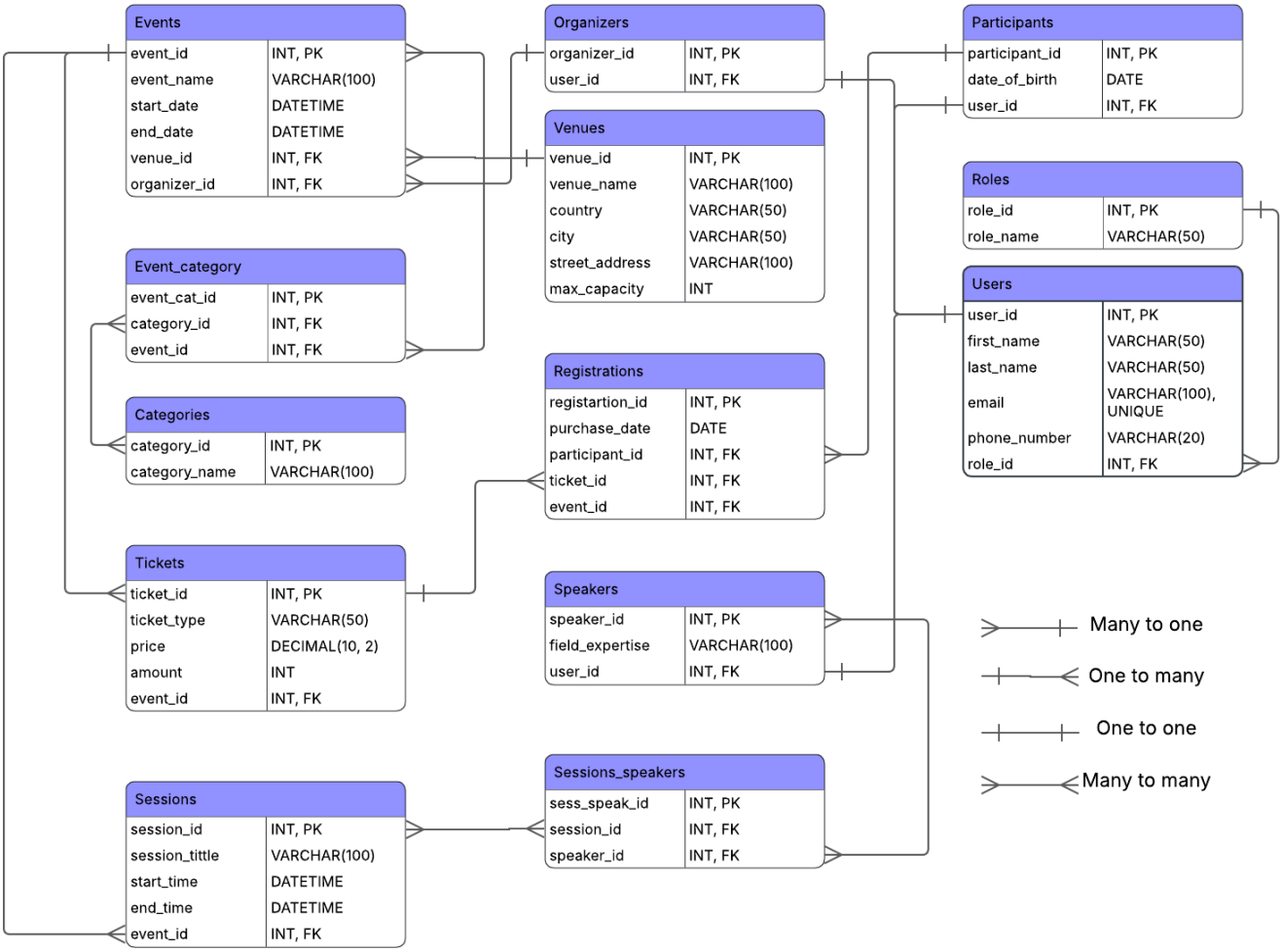


Figure 1: The Entity Relationship Diagram of the Event Management System database

3.0 Implementation

The database was implemented using SQL. Primary keys and foreign keys were used to enforce relationships between tables, foreign keys are used to ensure referential integrity. The implementation consists of 13 tables.

```
CREATE DATABASE EventManagementDB;
```
→ used to create a database

USE EventManagementDB;→ used to select the database

Creation of the tables:

3.1 Roles

```
CREATE TABLE Roles(
role_id INT AUTO_INCREMENT PRIMARY KEY,
role_name VARCHAR(50) NOT NULL);
```

The Roles table stores role_id as the primary key and role_name as a required field, which contains such values as: admin, organizer, participant or speaker. This table can be referenced by other tables, such as Users via primary key, to manage user permissions and access levels.

3.2 Users

```
CREATE TABLE Users(
user_id INT AUTO_INCREMENT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
email VARCHAR(100) NOT NULL UNIQUE,
phone_number VARCHAR(20) NOT NULL,
role_id INT,
FOREIGN KEY (role_id) REFERENCES Roles(role_id));
```

This table stores personal information about all users of the system. It includes user_id as the primary key, personal details such as name, email, and phone number, and a role_id foreign key that references the Roles table to assign a role to each user.

3.3 Organizers

```
CREATE TABLE Organizers(
organizer_id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT,
FOREIGN KEY (user_id) REFERENCES Users(user_id));
```

This table stores information about users who organize events. It contains organizer_id as the primary key and user_id as a foreign key, which links each organizer to a record in the Users table.

3.4 Participants

```
CREATE TABLE Participants(
participant_id INT AUTO_INCREMENT PRIMARY KEY,
date_of_birth DATE,
user_id INT,
FOREIGN KEY (user_id) REFERENCES Users(user_id));
```

This table stores information about users who attend events and includes participant_id as the primary key, date_of_birth as an additional attribute and user_id as a foreign key referencing the Users table.

3.5 Speakers

```
CREATE TABLE Speakers(
speaker_id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT,
field_expertise VARCHAR(100) NOT NULL,
FOREIGN KEY (user_id) REFERENCES Users(user_id));
```

This table stores information about event speakers. It contains speaker_id as the primary key, field_expertise to record the speaker's area of expertise, and user_id as a foreign key linking to the Users table.

3.6 Venues

```
CREATE TABLE Venues(
venue_id INT AUTO_INCREMENT PRIMARY KEY,
venue_name VARCHAR(100) NOT NULL,
country VARCHAR(50),
city VARCHAR(50),
street_address VARCHAR(100),
max_capacity INT);
```

The table stores information about locations where events are held and includes venue_id as the primary key, venue details such as name, country, city, street address, and maximum capacity.

3.7 Events

```
CREATE TABLE Events(
event_id INT AUTO_INCREMENT PRIMARY KEY,
event_name VARCHAR(100) NOT NULL,
start_date DATETIME NOT NULL,
end_date DATETIME NOT NULL,
venue_id INT,
organizer_id INT,
FOREIGN KEY (venue_id) REFERENCES Venues(venue_id),
FOREIGN KEY (organizer_id) REFERENCES Organizers(organizer_id));
```

The table stores information about events. It contains event_id as the primary key, event details such as name and schedule, and foreign keys referencing the Venues and Organizers tables to associate each event with a venue and organizer.

3.8 Categories

```
CREATE TABLE Categories(
category_id INT AUTO_INCREMENT PRIMARY KEY,
category_name VARCHAR(100) NOT NULL);
```

The table stores different categories of events, it contains category_id as the primary key and category_name as a required field storing the category name.

3.9 Sessions

```
CREATE TABLE Sessions(
session_id INT AUTO_INCREMENT PRIMARY KEY,
session_title VARCHAR(100) NOT NULL,
start_time DATETIME NOT NULL,
end_time DATETIME NOT NULL,
event_id INT,
FOREIGN KEY (event_id) REFERENCES Events(event_id));
```

This table stores information about sessions that belong to events. It includes session_id as the primary key, session title and schedule information, and event_id as a foreign key, which links each session to an event in the Events table.

3.10 Tickets

```
CREATE TABLE Tickets(
ticket_id INT AUTO_INCREMENT PRIMARY KEY,
ticket_type VARCHAR(50) NOT NULL,
price DECIMAL(10,2) NOT NULL,
amount INT NOT NULL,
event_id INT,
FOREIGN KEY (event_id) REFERENCES Events(event_id));
```

The table stores information about tickets for events. It contains ticket_id as the primary key, ticket type, price, available quantity, and event_id as a foreign key linking tickets to specific events.

3.11 Registrations

```
CREATE TABLE Registrations(
registration_id INT AUTO_INCREMENT PRIMARY KEY,
purchase_date DATE NOT NULL,
participant_id INT,
ticket_id INT,
event_id INT,
FOREIGN KEY (participant_id) REFERENCES Participants(participant_id),
FOREIGN KEY (ticket_id) REFERENCES Tickets(ticket_id),
FOREIGN KEY (event_id) REFERENCES Events(event_id));
```

It stores participant registrations for events and includes registration_id as the primary key, purchase date information, and foreign keys referencing the Participants, Tickets, and Events tables to record event attendance and ticket purchases.

3.12 Event_category

```
CREATE TABLE Event_category(
event_cat_id INT AUTO_INCREMENT PRIMARY KEY,
event_id INT,
category_id INT,
```

```
FOREIGN KEY (event_id) REFERENCES Events(event_id),
FOREIGN KEY (category_id) REFERENCES Categories(category_id));
```

This table serves as a junction table between events and categories. It contains event_cat_id as the primary key and foreign keys referencing the Events and Categories tables, which allows an event to belong to multiple categories and a category to be associated with multiple events.

3.13 Session_speaker

```
CREATE TABLE Session_speaker(
session_speak_id INT AUTO_INCREMENT PRIMARY KEY,
session_id INT,
speaker_id INT,
FOREIGN KEY (session_id) REFERENCES Sessions(session_id),
FOREIGN KEY (speaker_id) REFERENCES Speakers(speaker_id));
```

This table serves as a junction table between sessions and speakers and it includes session_speak_id as the primary key and foreign keys referencing the Sessions and Speakers tables, which allows multiple speakers to participate in a session and a speaker to present in multiple sessions.

After the creation of a database, it was filled with data to show how it might work in a real-world environment. To do so 4 roles (participant, organizer, speaker, admin) were added, 25 users (which were allocated to the roles above), 6 venues, 15 events, 10 categories, several sessions, tickets and registrations.

It is also important to mention that the database is normalized to Third Normal Form (3NF), which means that all repeating attributes are removed, many-to-many relationships are implemented using tables: Event_Category and Session_Speaker, and non-key attributes depend only on their primary keys. This reduces redundancy and improves data integrity.

4.0 Results

Query 1: Display registrations for the events

Input:

```
SELECT U.first_name, U.last_name, E.event_name, R.purchase_date
FROM Registrations R
JOIN Participants P ON R.participant_id = P.participant_id
JOIN Users U ON P.user_id = U.user_id
JOIN Events E ON R.event_id = E.event_id;
```

Output:

	first_name	last_name	event_name	purchase_date
1	Carol	Brown	AI Future Summit	2026-06-01
2	Carol	Brown	Future of Robotics Tokyo	2026-06-11
3	Tom	Shelter	AI Future Summit	2026-06-01
4	Tom	Shelter	Dubai Innovation Summit	2026-06-12
5	Liv	Mel	Berlin Startup Expo	2026-06-02
6	Liv	Mel	Middle East Tech Forum	2026-06-13
7	Mia	Taylor	Berlin Startup Expo	2026-06-02
8	Mia	Taylor	FinTech Dubai Expo	2026-06-14
9	Noah	Martin	Munich Tech Conference	2026-06-03
10	Noah	Martin	Global Cybersecurity Summit	2026-06-15
11	Sophia	Clark	San Francisco AI Hackathon	2026-06-04
12	Sophia	Clark	Cloud & AI World	2026-06-16
13	Liam	Walker	San Francisco AI Hackathon	2026-06-04
14	Liam	Walker	Cloud & AI World	2026-06-17
15	Olivia	Hall	Silicon Valley Meetup	2026-06-05
16	Ethan	Young	Warsaw Business Forum	2026-06-06
17	Ava	King	Eastern Europe Startup Day	2026-06-07
18	Lucas	Wright	Tokyo AI Conference	2026-06-08
19	Isabella	Scott	Tokyo AI Conference	2026-06-09
20	James	Adams	Japan Tech Expo	2026-06-10

This query by joining Participants, Users, Registrations and Events tables retrieves participant registration information and displays the participant's name, the event they registered for, and the purchase date. This query is used to track which participants and when were registered for specific events.

Query 2: Count registrations per events

Input:

```
SELECT E.event_name, COUNT(R.registration_id) AS Total_Registrations
FROM Events E
LEFT JOIN Registrations R ON E.event_id = R.event_id
```

```
GROUP BY E.event_name;
```

Output:

	event_name	Total_Registrations
1	AI Future Summit	2
2	Berlin Startup Expo	2
3	Cloud & AI World	2
4	Dubai Innovation Summit	1
5	Eastern Europe Startup Day	1
6	FinTech Dubai Expo	1
7	Future of Robotics Tokyo	1
8	Global Cybersecurity Summit	1
9	Japan Tech Expo	1
10	Middle East Tech Forum	1
11	Munich Tech Conference	1
12	San Francisco AI Hackathon	2
13	Silicon Valley Meetup	1
14	Tokyo AI Conference	2
15	Warsaw Business Forum	1

This query counts the total number of registrations for each event using the COUNT() aggregation function. This allows users to get information about the amount of people attending each event.

Query 3: Average ticket price per event

Input:

```
SELECT E.event_name, AVG(T.price) AS Average_Price
FROM Tickets T
JOIN Events E ON T.event_id = E.event_id
GROUP BY E.event_name;
```

Output:

	event_name	Average_Price
1	AI Future Summit	74.99
2	Berlin Startup Expo	74.995
3	Cloud & AI World	175
4	Dubai Innovation Summit	85
5	Eastern Europe Startup Day	70
6	FinTech Dubai Expo	110
7	Future of Robotics Tokyo	90
8	Global Cybersecurity Summit	120
9	Japan Tech Expo	75
10	Middle East Tech Forum	95
11	Munich Tech Conference	60
12	San Francisco AI Hackathon	140
13	Silicon Valley Meetup	45
14	Tokyo AI Conference	107.5
15	Warsaw Business Forum	55

This query calculates the average ticket price for each event using the AVG() function. Used to analyze the ticket pricing across events.

Query 4: Displays 5 events with most expensive tickets

Input:

```
SELECT E.event_name, T.ticket_type, T.price
FROM Tickets T
JOIN Events E ON T.event_id = E.event_id
ORDER BY price DESC
LIMIT 5;
```

Output:

	event_name	ticket_type	price
1	Cloud & AI World	VIP	250
2	San Francisco AI Hackathon	VIP	200
3	Tokyo AI Conference	VIP	150
4	Berlin Startup Expo	VIP	120
5	Global Cybersecurity Summit	Standard	120

This query shows 5 of the most expensive tickets for the events using LIMIT and ordering the prices in descending order. Helps to identify premium event offerings.

Query 5: Profit per event

Input:

```
SELECT E.event_name, SUM(T.price * T.amount) AS Profit
FROM Tickets T
JOIN Events E ON T.event_id = E.event_id
GROUP BY E.event_name
ORDER BY Profit  DESC;
```

Output:

	A-Z event_name	123 Profit
1	Cloud & AI World	57,000
2	Global Cybersecurity Summit	48,000
3	San Francisco AI Hackathon	40,000
4	FinTech Dubai Expo	38,500
5	Middle East Tech Forum	28,500
6	Future of Robotics Tokyo	27,000
7	Dubai Innovation Summit	23,800
8	Tokyo AI Conference	22,000
9	Japan Tech Expo	18,750
10	Eastern Europe Startup Day	15,400
11	Munich Tech Conference	12,000
12	AI Future Summit	9,998.5
13	Warsaw Business Forum	8,800
14	Silicon Valley Meetup	8,100
15	Berlin Startup Expo	7,199.2

This query calculates maximum profit/revenue by multiplying ticket price by tickets quantity and summing the results for each event. This helps to estimate the maximum revenue per event if all tickets are sold.

Query 6: Display the session titles with speakers and their fields of expertise

Input:

```
SELECT SE.session_title AS Session, CONCAT(U.first_name, ' ', U.last_name) AS Speaker,
SP.field_expertise AS Expertise
FROM Session_speaker SS
JOIN Sessions SE ON SS.session_id = SE.session_id
JOIN Speakers SP ON SS.speaker_id = SP.speaker_id
JOIN Users U ON SP.user_id = U.user_id
ORDER BY SE.session_title;
```

Output:

	Session	Speaker	Expertise
1	AI Hackathon Rules	Jake Green	Cloud Computing
2	AI Trends 2026	David Wilson	Artificial Intelligence
3	Cloud Infrastructure	Fred Duru	Cybersecurity
4	Cybersecurity Threats	David Wilson	Artificial Intelligence
5	Hackathon Coding Sprint	David Wilson	Artificial Intelligence
6	Investor Pitch Workshop	David Wilson	Artificial Intelligence
7	Machine Learning Basics	Fred Duru	Cybersecurity
8	Robotics Future	Jake Green	Cloud Computing
9	Startup Funding Guide	Jake Green	Cloud Computing
10	Startup Scaling	Fred Duru	Cybersecurity

This query joins the Sessions, Session_Speaker, Speakers, and Users tables to display session titles, speaker names using CONCAT(), and their fields of expertise. Tracks which speaker is assigned to each session.

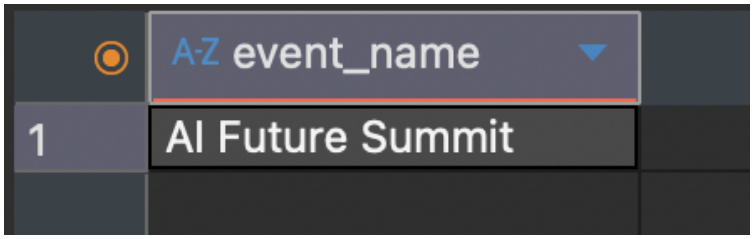
Query 7: Display the event with most registrations

Input:

```
SELECT event_name
FROM Events
WHERE event_id = (SELECT event_id
FROM Registrations
```

```
GROUP BY event_id
ORDER BY COUNT(*) DESC
LIMIT 1);
```

Output:



This query uses a subquery to identify the event with the greatest number of participant registrations. Used to determine the most popular event within the system.

Query 8: Transaction: ticket purchase

```
START TRANSACTION;
INSERT INTO Registrations
(purchase_date, participant_id, ticket_id, event_id)
VALUES
(CURDATE(), 8, 2, 1);
UPDATE Tickets
SET amount = amount - 1
WHERE ticket_id = 2;
COMMIT;
-- Checks whether the transaction works
ROLLBACK;
SELECT * FROM Tickets WHERE ticket_id = 2;
```

The transaction simulates the process of purchasing an event ticket, as it performs two operations at the same time, first one is creating a new registration for an existing participant and then it reduces the number of available tickets. Transaction is used to run both processes consistently until COMMIT, which maintains the data integrity, as if one process fails the other process won't start. To test the transaction ROLLBACK was used to ensure that all transaction changes can be declined until the COMMIT is called.

CRUD queries: (updating the price, deleting the registration, adding the registration)

```
-- CRUD queries
-- Checking the ticket price
SELECT ticket_id, ticket_type, price
FROM Tickets
WHERE ticket_id = 1;
-- Updating the ticket price
UPDATE Tickets
SET price = 75.00
WHERE ticket_id = 1;
-- Checking for the existence of the registration
SELECT *
FROM Registrations
WHERE registration_id = 5;
-- Deleting the registration
DELETE FROM Registrations
WHERE registration_id = 5;
-- Restoring/adding back the registration
INSERT INTO Registrations (registration_id, participant_id, event_id)
VALUES (5, 3, 2);
```

CRUD operations are implemented to demonstrate the database's ability to Create, Read, Update, and Delete records.

Read: the system retrieves ticket information before changes are made

Update: the system updates ticket price from its original value to a new value.

Delete: the system removes the registration from the Registrations table.

Create: A registration record is inserted back into the database.

5.0 Challenges and solutions

While developing the database for the events management system several challenges were encountered.

The first one is managing the relationships between users and other tables with specific roles. To solve this problem several tables were created: one for all the users and separate for participants, organizers and speakers, as this helps to avoid data duplication.

The second one was more as a simplification for the system not to join tables many times, but already have ready many-to-many relationship tables such as: Event_Category and Session_Speaker.

The third problem was that the system should ensure the data integrity to do so; primary keys, foreign keys, NOT NULL constraints, and UNIQUE constraints were used throughout the database.

6.0 Conclusion and future work

This project successfully developed a relational Event Management System using MySQL. The database stores and manages information related to users, organizers, participants, speakers, events, venues, sessions, tickets, and registrations. The implemented queries demonstrate the ability of the system to perform data retrieval, reporting, aggregation and filtering.

In future the system can be improved by adding more tables, for example, for handling information about online payments for participants and wages for organizers and speakers, the system can also be modified to allow admins and/or organizers track attendance for the events.

Overall, the project demonstrates the practical application of relational database concepts including normalization, constraints, relationships, joins, and SQL querying.